



WYŻSZA SZKOŁA ZARZĄDZANIA
I BANKOWOŚCI W KRAKOWIE

LABORATORIUM METODY OBLICZENIOWE I SYMULACJA

Patryk Płatek

Grupa: 2K332-PO

05.10.2025

LABORATORIUM 1: ARYTMETYKA KOMPUTEROWA

ZAWARTOŚĆ

Laboratorium 1: Arytmetyka komputerowa.....	1
zadania 1,2.....	3
Podejście do rozwiązania	3
Rozwiązanie	5
Wyniki	11
Wnioski.....	13
Zadanie 3	14
Podejście do rozwiązania	14
Rozwiązanie	14
Wyniki	16
Wnioski.....	16
Bibliografia.....	17

ZADANIA 1,2

1. Napisac program liczacy kolejne wyrazy ciagu: $x_{n+1} = x_n + 3x_n(1 - x_n)$ startujac z punktu $x_0 = 0.01$. Wykonac to zadanie dla roznych reprezentacji liczb (float, double). Dlaczego wyniki sie rozbiegaja?
2. Napisac program liczacy ciag z wczesniejszego zadania, ale wg wzoru $4x_n - 3x_n^2$ porownac z wynikami z wczesniejszego zadania.

PODEJŚCIE DO ROZWIĄZANIA

1. Wybór dokładnej arytmetyki referencyjnej

- Zdecydowałem się wykorzystać BigDecimal jako wartość referencyjną, ponieważ float i double w Javie wprowadzają błędy zaokrąglenia na każdym kroku iteracji.
- BigDecimal pozwala kontrolować precyzję i utrzymać niemal idealną reprezentację liczby w zakresie badanego przedziału, co jest kluczowe do porównania błędów w arytmetyce zmiennoprzecinkowej.
- Argument: bez wartości referencyjnej nie można było rzetelnie ocenić, która reprezentacja float/double jest bliższa „prawdziwej” trajektorii.

2. Abstrakcja i struktura kodu

- Utworzyłem klasę abstrakcyjną FloatingPointExperiment z metodami getNext() dla float, double i BigDecimal.
- Każda implementacja eksperymentu (np. Zadanie1, Zadanie2) definiuje tylko własną logikę funkcji iteracyjnej w tych trzech typach.
- Argumentacja:
 - Ułatwia rozszerzalność – kolejne funkcje iteracyjne można wprowadzić bez przepisywania całego mechanizmu porównania.
 - Zapewnia spójność porównań – wszystkie eksperymenty korzystają z tego samego mechanizmu iteracji, zapisu wyników i obliczania błędów.

3. Dokładny zapis wyników

- Każdy krok iteracji zapisywany jest do pliku CSV z:
 - wartością rzeczywistą (float/double),
 - reprezentacją hex (IEEE754),
 - wartością referencyjną BigDecimal,
 - błędem bezwzględnym (f_abs_err i d_abs_err).
- Argumentacja: zapis w hex umożliwia dokładne odwzorowanie bitowe float/double w Pythonie i późniejsze tworzenie precyzyjnych wykresów.
- Pozwala analizować skumulowany efekt błędów w iteracjach – można łatwo ocenić, w którym momencie float/double odchodzi od trajektorii BigDecimal.

4. Porównanie błędów i wizualizacja

- W Pythonie przygotowałem wykresy:
 - wartości x_n dla float, double i BigDecimal,
 - błędy bezwzględne dla float i double,
 - różnicę błędów między eksperymentami lub funkcjami $A(x)/B(x)$
- Wizualizacja logarytmiczna błędów umożliwia **wyraźne pokazanie momentów, w których float staje się nieprecyzyjny**, a także jak błędy się kumulują w kolejnych iteracjach.

5. Analiza pesymistyczna błędów

Obliczyłem ograniczenia pesymistyczne $A(x)$ i $B(x)$

- według wzorów teoretycznych.
- Argumentacja: pozwala zrozumieć teoretyczne maksimum błędu na krok, które zależy od precyzji i kolejności operacji arytmetycznych.
- W połączeniu z wynikami numerycznymi pozwala zobaczyć, w których przedziałach x jeden wariant funkcji jest bardziej „bezpieczny”.

ROZWIĄZANIE

$$A: x_{n+1} = x_n + 3x_n(1 - x_n)$$

$$B: x_{n+1} = 4x_n - 3x_n^2$$

Gdybyśmy mogli obliczać w dokładnej arytmetyce rzeczywistej, każde x_n byłoby dokładnym wynikiem f oba równania byłyby sobie równoznaczne:

$$A(x) = B(x)$$

W arytmetyce komputerowej wykonujemy kilka operacji elementarnych, każda ze swoim błędem:

$$fl(a \text{ op } b) = (a \text{ op } b)(1 + \varepsilon), \quad |\varepsilon| \leq u$$

Gdzie unit roundoff:

$$u = \frac{1}{2}\beta^{1-t}$$

Podczas iteracyjnych obliczeń każda operacja w arytmetyce zmiennoprzecinkowej wprowadza pewien błąd lokalny. Całkowity błąd w n -tej iteracji można rozdzielić na część wynikającą z niedokładności obliczeń oraz na propagowany błąd danych wejściowych.

“Denote the true value of the input data by x , so that the desired true result is $f(x)$. Suppose that we must work with inexact input, say $\hat{x}$, and we can compute only an approximation to the function, say $\hat{f}$. Then

$$\text{Total error} = \hat{f}(\hat{x}) - f(x)$$

$$= (\hat{f}(\hat{x}) - f(\hat{x})) + (f(\hat{x}) - f(x))$$

$$= \text{computational error} + \text{propagated data error.}”$$

- Heath, M. T.

W naszym przypadku `computational error` odpowiada błędom w pojedynczych krokach iteracji wynikającym z ograniczonej precyzji typu `float` lub `double`, natomiast `propagated data error` uwzględnia skumulowane błędy z poprzednich iteracji.

POSTAĆ A $x_n + 3x_n(1 - x_n)$ (ROZPISANIE OPERACJI)

Równanie $x_n + 3x_n(1 - x_n)$ w arytmetyce komputerowej składa się z kilku operacji:

$$\begin{aligned}
 t_1 &= fl(1 - x_n) = (1 - x_n)(1 + \varepsilon_1) \\
 t_2 &= fl(3x_n) = (3x_n)(1 + \varepsilon_2) \\
 t_3 &= fl(t_2 t_1) = (t_2 t_1)(1 + \varepsilon_3) \\
 \hat{x}_{n+1} &= fl(x_n + t_3) = (x_n + t_3)(1 + \varepsilon_4) \\
 \tilde{x}_{n+1} &= (x_n + 3x_n(1 - x_n)(1 + \varepsilon_1)(1 + \varepsilon_2)(1 + \varepsilon_3))(1 + \varepsilon_4)
 \end{aligned}$$

Aproksymacja pierwszego rzędu (pomijamy $O(\varepsilon^2)$)

$$\tilde{x}_{n+1} \approx f(x) + f(x)\varepsilon_4 + 3x_n(1 - x_n)(\varepsilon_1 + \varepsilon_2 + \varepsilon_3)$$

Lokalny błąd kroku:

$$\eta_n \approx \tilde{x}_{n+1} - f(x) \approx f(x)\varepsilon_4 + 3x_n(1 - x_n)(\varepsilon_1 + \varepsilon_2 + \varepsilon_3)$$

Pesymistycznie, jeśli wszystkie ε_i mają ten sam znak i $|\varepsilon_i| \leq u$,

$$|\varepsilon_1 + \varepsilon_2 + \varepsilon_3| \leq 3u$$

$$\begin{aligned}
 |\eta_n| &\leq 3 |x_n(1 - x_n)| \cdot 3u + |f(x_n)| \cdot u \\
 |\eta_n| &\leq u(|f(x_n)| + 9 |x_n(1 - x_n)|)
 \end{aligned}$$

Dla $x \in [0,1]$ gdzie $f(x) = 4x - 3x^2 \geq 0$ więc możemy napisać bez wartości bezwzględnej:

$$A(x) = \frac{|\eta_n|}{u} \leq 4x - 3x^2 + 9x(1 - x) = 13x - 12x^2$$

POSTAĆ B $4x_n - 3x_n^2$ (ROZPISANIE OPERACJI)

Dla uproszczonej wersji równania $x_{n+1} = 4x_n - 3x_n^2$

$$t_1 = fl(4x_n) = (4x_n)(1 + \varepsilon_1)$$

$$t_2 = fl(x_n^2) = (x_n^2)(1 + \varepsilon_2)$$

$$t_3 = fl(3x_n) = (3x_n^2)(1 + \varepsilon_3)$$

$$\hat{x}_{n+1} = fl(t_1 - t_3) = (t_1 - t_3)(1 + \varepsilon_4)$$

$$\tilde{x}_{n+1} \approx (4x_n - 3x_n^2) + 4x_n\varepsilon_1 - 3x_n^2(\varepsilon_2 + \varepsilon_3) + (4x_n - 3x_n^2)\varepsilon_4$$

Lokalny błąd w pojedynczym kroku:

$$\eta_n \approx \tilde{x}_{n+1} - x_n \approx 4x_n\varepsilon_1 - 3x_n^2(\varepsilon_2 + \varepsilon_3) + (4x_n - 3x_n^2)\varepsilon_4$$

pesymistyczne ograniczenie:

$$|\eta_n| \leq u(4|x_n| + 6|x_n^2| + |4x_n - 3x_n^2|)$$

Dla $x \in [0,1]$ mamy $f(x) = 4x - 3x^2 \geq 0$ więc możemy napisać bez wartości bezwzględnej:

$$B(x) = \frac{|\eta_n|}{u} \leq 4x + 6x^2 + 4x - 3x^2 = 8 + 3x^2$$

PORÓWNANIE ALGEBRAICZNE $A(x)$, $B(x)$

Obliczamy różnicę:

$$A(x) - B(x) = (13x - 12x^2) - (8x + 3x^2) = 5x(1 - 3x)$$

- Dla $x < \frac{1}{3}$, $A(x) - B(x) > 0 \rightarrow A(x) > B(x)$
B ma mniejsze pesymistyczne ograniczenie.
- Dla $x > \frac{1}{3}$, $A(x) - B(x) < 0 \rightarrow A(x) < B(x)$
A ma mniejsze pesymistyczne ograniczenie.
- Dla $x = \frac{1}{3}$ obie formy są równoważne pod względem górnego ograniczenia.

INTERPRETACJA

- Gdy x jest małe (np. 0.01): uproszczona postać B : $x_{n+1} = 4x_n - 3x_n^2$ ma mniejsze pesymistyczne ograniczenie błędu na krok teoretycznie jest „bezpieczniejsza”.
- Gdy x rośnie powyżej $\frac{1}{3}$ postać A : $x_{n+1} = x_n + 3x_n(1 - x_n)$ ma mniejsze górne ograniczenie.

UWAGI

To są *pesymistyczne* oszacowania zakładają, że wszystkie ε_i mają ten sam znak i maksymalną wartość. W praktyce rzeczywiste błędy zwykle są mniejsze.

Równanie B kończy się operacją odejmowania $t_1 - t_3$ jeżeli wartości są bliskie, może wystąpić *katastrofalne anulowanie* (względny błąd gwałtownie rośnie).

W postaci A występuje dodawanie, więc to ryzyko jest mniejsze.

Analiza taka zakłada, że poprzednia wartość x_n jest dokładna. W rzeczywistości jednak, ponieważ badana funkcja jest rekurencyjna błędy powstałe w wcześniejszych krokach są przenoszone i akumulują się w kolejnych iteracjach.

Powstaje błąd globalny:

$$e_n = \tilde{x}_n - \tilde{x}_n$$

czyli rzeczywistej różnicy pomiędzy wartością obliczoną a wartością dokładną po n krokach.

$$e_{n+1} \approx f'(x)e_n + \eta_n$$

co oznacza:

- poprzedni błąd e_n jest wzmacniany współczynnikiem równym pochodnej funkcji $f'(x)$,
- do tego dodawany jest nowy lokalny błąd η_n .

Dla obu badanych postaci funkcji:

$$f'(x) = 4 - 6x$$

Zatem propagacja błędów w obu przypadkach przebiega podobnie, natomiast różnice w dokładności wynikają z różnych zestawów operacji arytmetycznych w poszczególnych krokach, które prowadzą do odmiennych wartości lokalnych błędów η_n

Dlatego wyniki dla typu float i double są różne:

“unit in the last place’ (ulp) depends on the precision of the floating-point format in use. For a nonzero finite base- β string it is the magnitude of its least significant digit, or in other words, the distance between the floating point number and the next floating point number of greater magnitude.”

— M. Möller.

Każda iteracja powoduje błąd zaokrąglenia wynoszący około $\frac{1}{2}ulp(x_n)$.

W przypadku pojedynczej precyzji (float) *ulp* jest znacznie większy, więc błędy kumulują się szybciej.

W przypadku podwójnej precyzji (double) *ulp* jest mniejszy, więc sekwencja pozostaje bliższa rzeczywistej trajektorii przez większą liczbę iteracji.

typ	precyzja	$ulp(1)$	$ulp(0.01)$
float	24	1.192×10^{-7}	9.31×10^{-10}
double	53	2.22×10^{-16}	1.73×10^{-18}

W celu przeprowadzenia porównania wyników obliczeń dla różnych typów zmiennopozycyjnych (float, double) oraz wartości referencyjnych (BigDecimal), zaprojektowano hierarchię klas z wykorzystaniem klasy abstrakcyjnej oraz klas ją implementujących.

Zawiera ona wspólne elementy logiki, takie jak:

- zarządzanie liczbą iteracji (maxIter),
- wykonywanie pętli obliczeniowej,
- zapis wyników do pliku CSV

Definiuje również abstrakcyjne metody:

```
public abstract double getNext(double x_n);
public abstract float getNext(float x_n);
// wartość referencyjna
public abstract BigDecimal getNext(BigDecimal x_n);
```

$$x_{n+1} = x_n + 3x_n(1 - x_n)$$

```
@Override
public double getNext(double x_n) {
    return x_n + 3.0d * x_n * (1d - x_n);
}

@Override
public float getNext(float x_n) {
    return x_n + 3.0f * x_n * (1f - x_n);
}

@Override
public BigDecimal getNext(BigDecimal x_n) {
    final MathContext MC = MathContext.DECIMAL128;
    // 1 - x_n
    BigDecimal t1 = BigDecimal.ONE.subtract(x_n, MC);
    // 3 * x_n
    BigDecimal t2 = BigDecimal.valueOf(3).multiply(x_n, MC);
    // 3 * x_n * (1 - x_n)
    BigDecimal t3 = t2.multiply(t1, MC);
    // x_n + 3 * x_n * (1 - x_n)
    return t3.add(x_n, MC);
}
```

$$x_{n+1} = 4x_n - 3x_n^2$$

```
@Override
public double getNext(double x_n) {
    return 4.0 * x_n - (3 * (x_n * x_n));
}

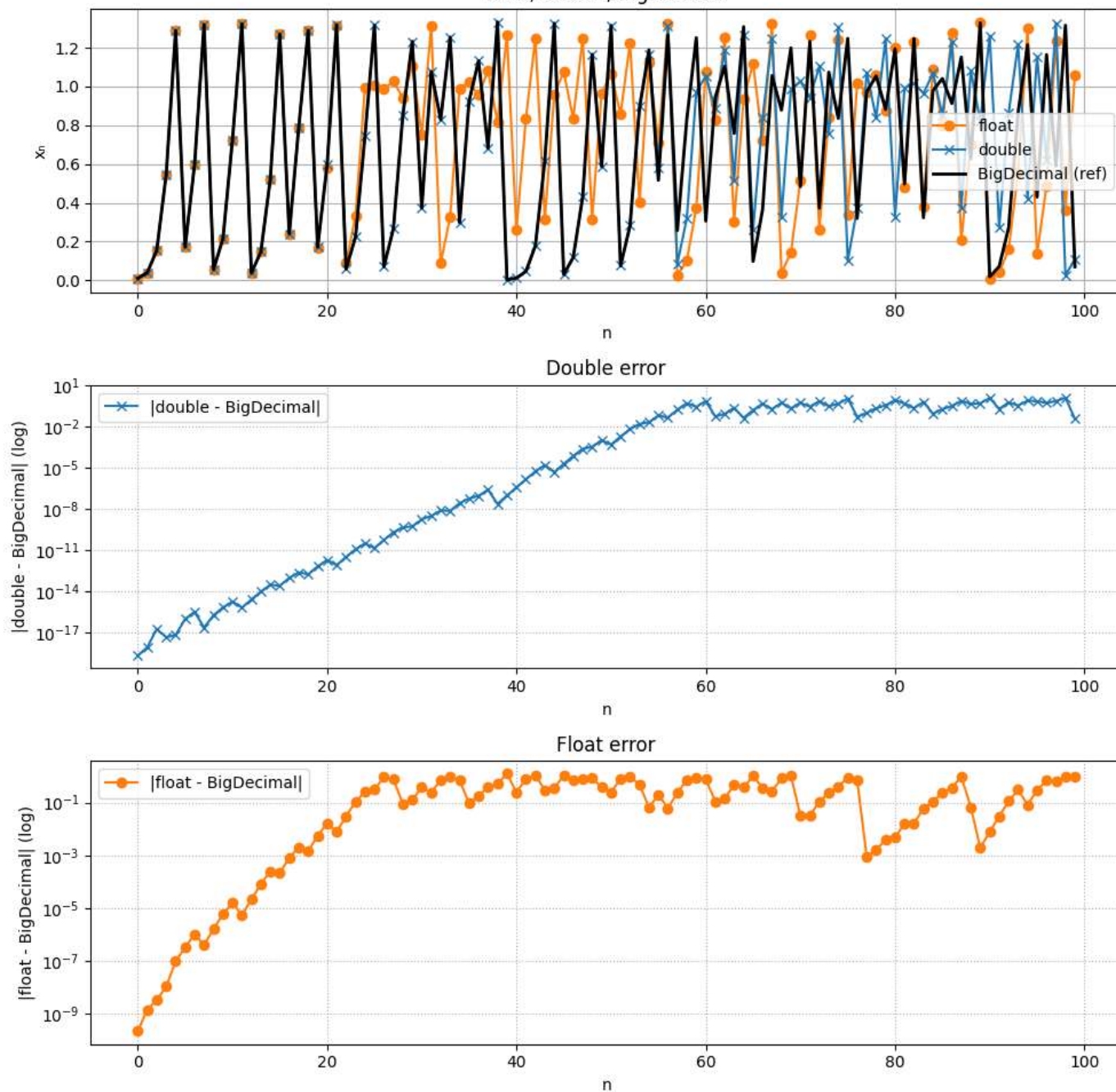
@Override
public float getNext(float x_n) {
    return 4.0f * x_n - (3f * (x_n * x_n));
}

@Override
public BigDecimal getNext(BigDecimal x_n) {
    final MathContext MC = MathContext.DECIMAL128;
    // 4 * x_n
    BigDecimal t1 = BigDecimal.valueOf(4).multiply(x_n, MC);
    // x_n^2
    BigDecimal t2 = x_n.multiply(x_n, MC);
    // 3 * x_n^2
    BigDecimal t3 = BigDecimal.valueOf(3).multiply(t2, MC);
    // 4 * x_n - 3 * x_n^2
    return t1.subtract(t3, MC);
}
```

WYNIKI

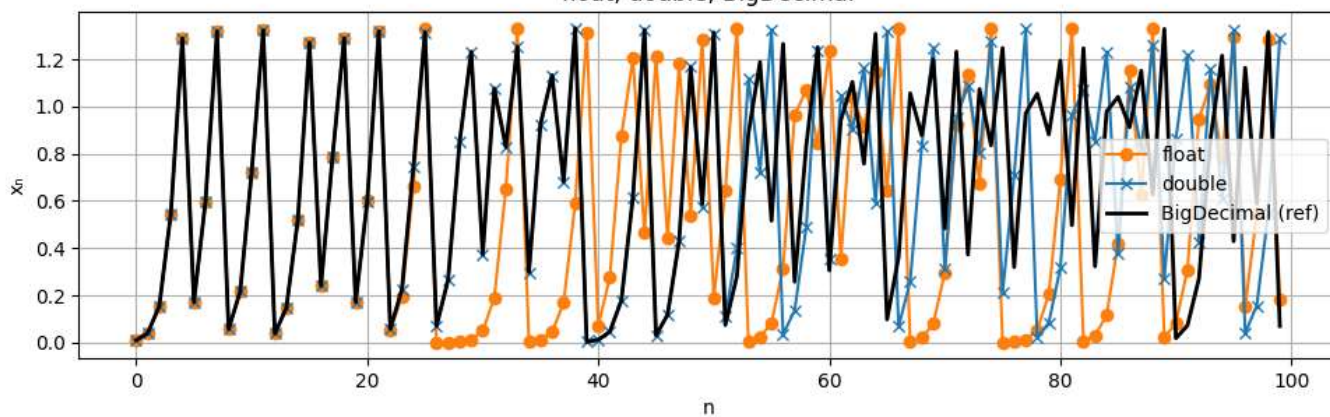
$$x_{n+1} = x_n + 3x_n(1 - x_n)$$

float, double, BigDecimal

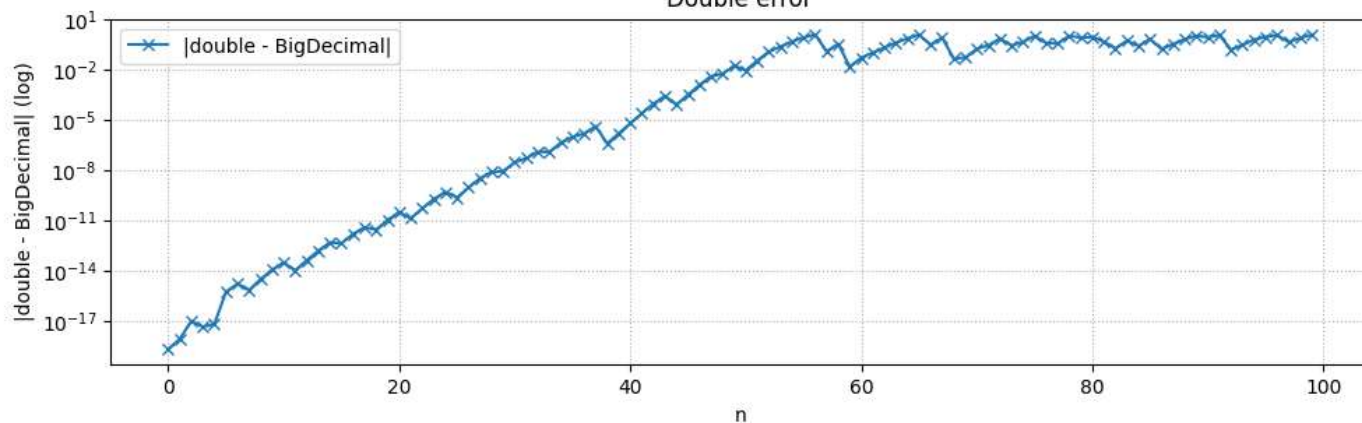


$$x_{n+1} = 4x_n - 3x_n^2$$

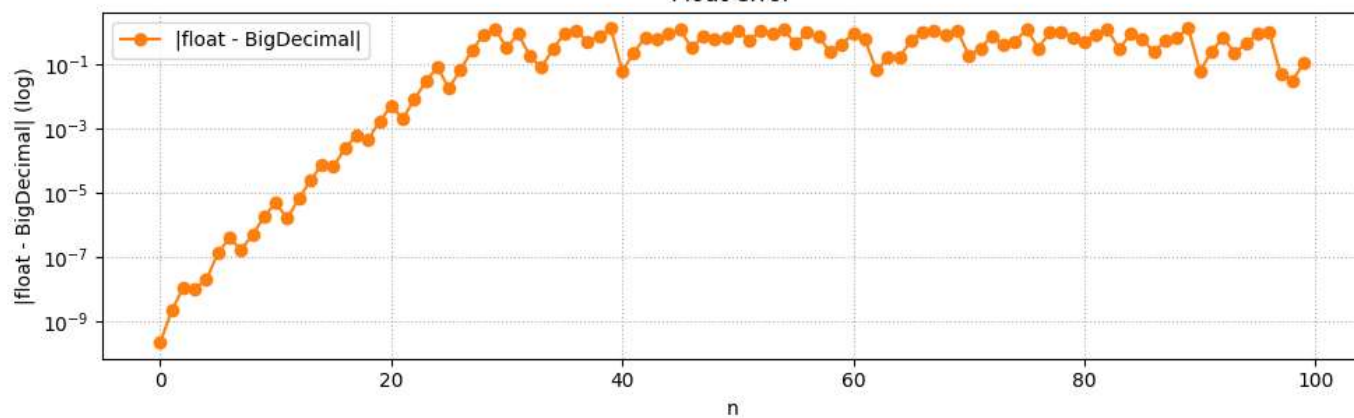
float, double, BigDecimal



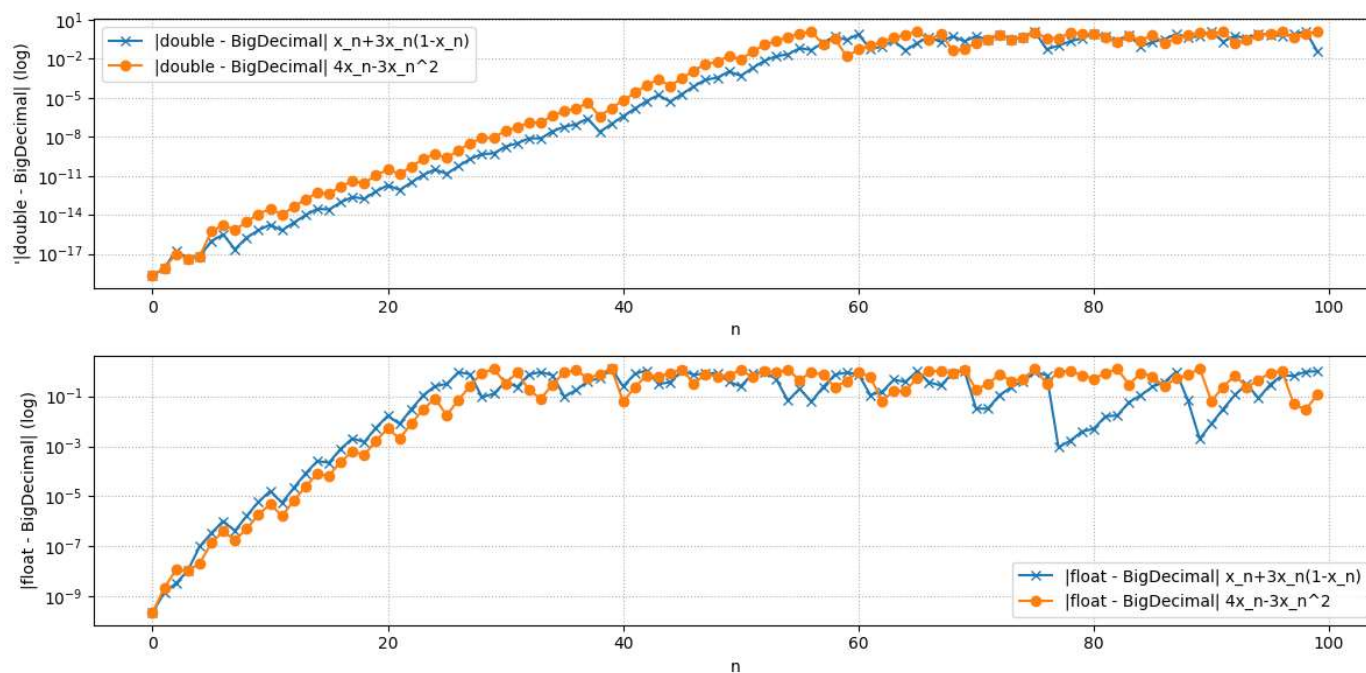
Double error



Float error



Porównanie błędów



WNIOSKI

1. Różnice float vs double – Float ma mniejszą precyzję i większy *ulp*, więc błędy kumulują się szybciej. Double utrzymuje dokładność przez więcej iteracji.
2. BigDecimal jako referencja – Umożliwia dokładne określenie błędu każdego kroku i wizualizację odchyleń float/double.
3. Błąd skumulowany – W iteracjach rekurencyjnych lokalne błędy propagują się: $e_{n+1} \approx f'(x)e_n + \eta_n$
4. Praktyczna obserwacja – Float jest szybszy, ale mniej precyzyjny; double bardziej precyzyjny; BigDecimal dokładny, ale wolny.

ZADANIE 3

Znaleźć "maszynowe epsilon", czyli najmniejszą liczbę a , taką że $a + 1 > 1$

PODEJŚCIE DO ROZWIĄZANIA

Iteracyjne zmniejszanie epsilon

Algorytm startuje od $\epsilon = 1.0$. W każdej iteracji epsilon dzielony jest przez 2 i sprawdzane jest, czy $1 + \epsilon/2$ nadal różni się od 1.0.

- Jeśli tak, epsilon jest dalej zmniejszany.
- Jeśli nie, ostatnia wartość epsilon, która powodowała zmianę, jest zwracana jako maszynowy epsilon.

Argumentacja: metoda ta wykorzystuje fakt, że w reprezentacji zmiennoprzecinkowej istnieje minimalny krok, którego komputer jest w stanie „wyczuć” przy dodawaniu do 1.0.

ROZWIĄZANIE

```
private static float findMachineEpsilonFloat() {  
    float epsilon = 1.0f;  
    while (Float.compare((1.0f + epsilon / 2.0f), 1.0f) > 0) {  
        epsilon = epsilon / 2.0f;  
    }  
    return epsilon;  
}
```

Opis działania:

- Na początku epsilon ustawiane jest na wartość $1.0f$.
- W pętli wykonywana jest operacja dodawania $1.0f + \epsilon/2.0f$.
- Jeśli wynik tej operacji jest większy od 1.0, to znaczy, że komputer potrafi rozróżnić te dwie liczby — zmniejszamy więc epsilon o połowę.
- Gdy w pewnym momencie $1.0f + \epsilon/2.0f$ nie różni się już od $1.0f$ (ze względu na ograniczoną precyzję typu float), pętla kończy się.
- Ostatnia wartość epsilon, która jeszcze powodowała zmianę, zostaje zwrócona jako maszynowy epsilon.

Wynik:

Dla typu float w standardzie IEEE 754: $\approx 1.1920929 \times 10^{-7}$

```
private static double findMachineEpsilonDouble() {  
    double epsilon = 1.0;  
    while (Double.compare(1.0 + epsilon / 2.0, 1.0) > 0) {  
        epsilon = epsilon / 2.0;  
    }  
    return epsilon;  
}
```

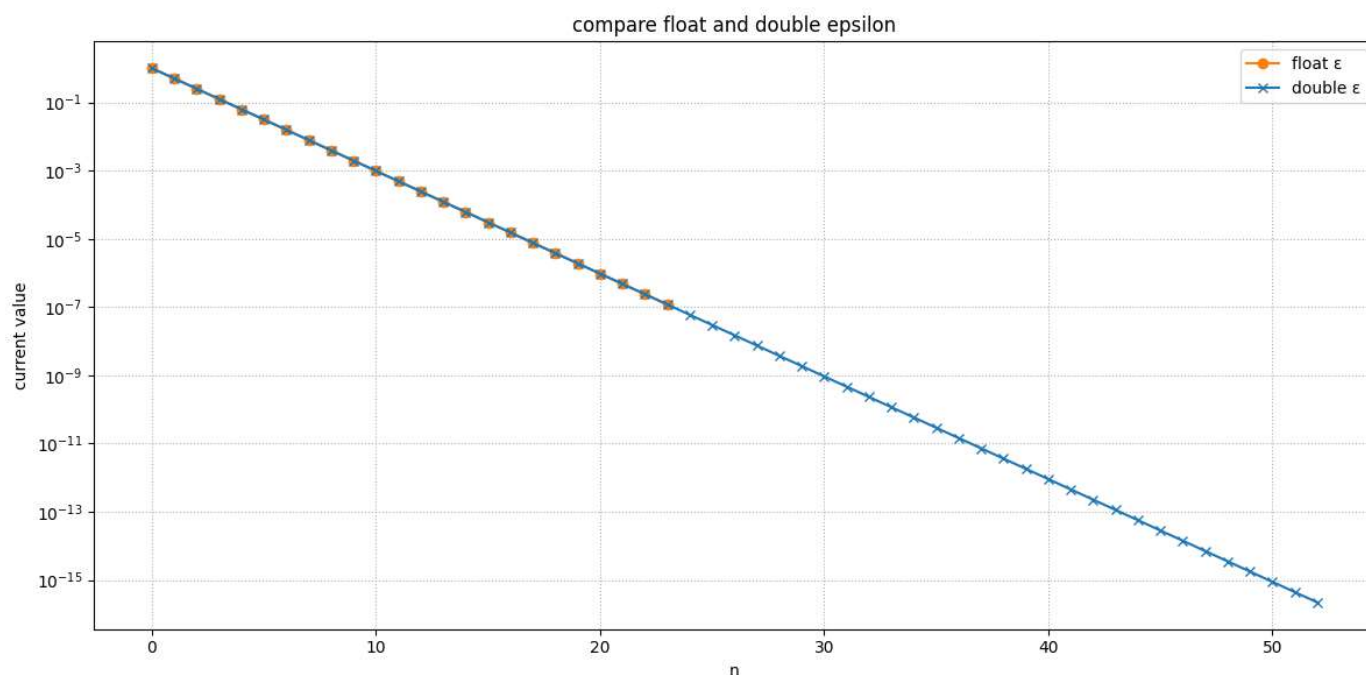
Opis działania:

- Działa dokładnie tak samo jak wersja dla float, ale wykorzystuje typ double, który ma większą precyzję (ok. 15–16 cyfr znaczących).

Wynik: Dla typu double w standardzie IEEE 754: $\approx 2.220446049250313 \times 10^{-16}$

WYNIKI

- dla float błąd względny $\approx 1.1920929 \times 10^{-7}$
- dla double błąd względny $\approx 2.220446049250313 \times 10^{-16}$



INTERPRETACJA WYNIKÓW

Maszynowy epsilon określa granice dokładności obliczeń zmiennoprzecinkowych. Każda operacja arytmetyczna w komputerze obciążona jest błędem rzędu ϵ .

WNIOSKI

Maszynowy epsilon dla danego typu zmiennoprzecinkowego określa minimalny krok, który komputer jest w stanie „wyczuć” przy dodawaniu do 1.0. Wartość epsilon jest zależna od precyzji typu danych: float ma większy epsilon od double co przekłada się na dokładność obliczeń. Metoda iteracyjnego dzielenia epsilon przez 2 jest prostym, ale skutecznym sposobem na określenie tej minimalnej rozróżnialnej wartości dla różnych typów zmiennoprzecinkowych.

BIBLIOGRAFIA

Heath, M. T. *SCIENTIFIC COMPUTING An Introductory Survey*.

Muller, J.-M. *Elementary Functions Algorithms and Implementation Third Edition*. Birkhäuser.

Repozytorium projektu:

Kod źródłowy oraz wyniki eksperymentów dostępne są w repozytorium GitHub:

https://github.com/SaberLS/calculation_methods_and_simulation_lab